



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程设计实验报告

实验三：操作系统应用程序开发——走迷宫

院系： 计算机科学与技术学院
专业： 计算机科学与技术
姓名： 毛星博
学号： 2351520
指导老师： 郭玉臣
完成时间： 2026年6月28日

1、实验目的

- 本实验基于已经成功移植到 OpenMIPS 平台上的 μ C/OS-II 操作系统，完成一个能够在串口终端中运行的可选难度走迷宫小游戏。程序不再只是简单输出字符串，而是围绕真实交互过程组织输入、状态、显示和重启逻辑，因此能够更完整地体现嵌入式应用开发中软硬件协同工作的特点。
- 锻炼系统级软件的分析与调试能力，通过实践，培养在没有图形化界面的嵌入式环境下，分析、定位和解决从硬件初始化、系统启动到应用程序运行等各个环节问题的综合能力。
- 加强对计算机专业核心课程的掌握，融合编译原理、操作系统等底层软件知识与硬件课程，构建软硬件一体化的完整知识架构。通过本实验，希望掌握 UART 原始字符输入输出、ANSI 终端控制、帧缓冲渲染、GPIO 与键盘双输入模式、状态转换设计以及 μ C/OS-II 用户任务组织方式。相比普通 PC 程序，这些内容更贴近底层系统实验，也更能训练在资源受限环境中拆解问题和调试程序的能力。

二、实验内容

(1) 实验环境与硬件配置

- 处理器： 13th Gen Intel(R) Core(TM) i9-13900HX
- 操作系统： Windows 11
- 开发平台： Vivado 2022.2
- Linux 虚拟机： VMware Ubuntu22.04
- 串口通信工具： PUTTY
- 开发板： NEXYS4DDRAtrix-7

使用 PuTTY 而不是指导书上的 `sscom` 的原因在于走迷宫游戏需要频繁刷新同一块终端区域，如果每走一步都重新打印整张地图，终端窗口会不断滚动，既不美观，也不符合游戏界面的直觉。PuTTY 能正确处理 ANSI 转义序列，例如清屏、光标定位和隐藏光标。程序因此可以把输出从“追加文本”改为“原地更新”：首帧清屏绘制，后续帧只把发生变化的行定位到原位置重新输出。这样一来，玩家看到的是地图局部变化，而不是一屏又一屏重复的迷宫。

项目	说明
操作系统工程	OpenMIPS + μ C/OS-II，用户程序位于 ucosii_OpenMIPS/common/openmips.c。
开发与构建	VSCode 修改源码，课程 Makefile 完成交叉编译，生成 OS.bin。
FPGA 与烧录	Vivado 完成 bit 流下载、SPI-Flash 配置与程序写入。
串口终端	PuTTY 支持 ANSI 光标定位与原地刷新，更适合迷宫差分渲染。
输入方式	保留开发板 GPIO 开关/N17 输入，同时新增键盘输入模式。
调试目标	观察菜单选择、差分渲染、键盘移动、撞墙提示、通关重启是否符合预期。

（2）功能目标

走迷宫小游戏需要实现以下功能：第一，支持 Easy、Medium、Hard、Hell 四种难度地图；第二，保留开发板 GPIO 输入方式；第三，新增键盘输入方式，使玩家可以通过 PuTTY 直接按键操作；第四，使用帧缓冲和差分渲染优化终端显示；第五，明确表达游戏中、撞墙提示、成功三个核心状态之间的转换关系。

这些功能使程序从一个“串口输出型演示”进一步变成一个“终端交互型应用”。用户既可以使用实验板开关体验硬件输入，也可以使用键盘完成更自然的游戏控制；显示层也从简单重画升级为有状态的局部刷新。

（3）总体架构

代码可以分为五个层次：底层 UART/GPIO/定时器初始化层，UART 原始输入输出层，帧缓冲渲染层，迷宫数据与状态层，以及 TaskStart 主任务控制层。底层初始化保证程序能够运行；原始 UART 层负责 PuTTY 的键盘输入和 ANSI 输出；帧缓冲层负责构造并渲染界面；迷宫层保存地图、玩家坐标和出口坐标；主任务则把难度选择、游戏运行和通关重启串起来。



模块	作用	关键变量/函数
原始 UART 输出	发送 ANSI 序列与帧缓冲内容，不进行普通 uart_putc 的 CRLF 转换。	uart_putc_raw(), uart_send_str(), uart_ansi_pos()
键盘读取	非阻塞读取 UART 输入，解析 WASD、数字键、Enter、Space 和方向键转义序列。	uart_getc(), uart_get_key(), K_UP/K_CONFIRM 等
帧缓冲	在内存中构造新旧两帧字符画面，用于比较变化。	fb_new, fb_old, FB_ROWS, FB_COLS
差分渲染	首帧清屏全量绘制，后续只输出发生变化的行，减少终端滚屏和重绘。	fb_render(), fb_force_full_refresh()
界面构建	把菜单界面和游戏界面写入 fb_new，再交给渲染器输出。	build_difficulty_screen(), build_game_screen()

三、关键代码与实现分析

(1) 程序入口与任务创建

程序入口仍然保持课程工程原有结构，main 函数负责初始化 μ C/OS-II、UART 和 GPIO，然后创建 TaskStart 用户任务。真正的游戏逻辑都放在 TaskStart 内部执行。这样做的好处是应用层更新不会破坏底层启动流程，后续从 2048 改成迷宫、再从全屏重画改成差分渲染，都不需要改动主入口框架。

```
void main()
{
    OSInit(); /*  $\mu$ C/OS-II 初始化 */

    uart_init(); /* UART 控制器初始化 */
    gpio_init(); /* GPIO 模块初始化 */
```

```

/* 创建用户任务 */
OSTaskCreate(TaskStart, (void *)0, &TaskStartStk[TASK_STK_SIZE - 1], 0);
OSStart(); /* μC/OS-II 启动 */
}

```

(2) PuTTY 与原始 UART 输出

为了让 PuTTY 正确识别 ANSI 控制序列，我增加了原始 UART 输出函数。普通 `uart_putc()` 在输出换行时会自动补回车，这适合普通文本打印，但不适合发送 ESC 控制序列和帧缓冲内容。因此代码新增 `uart_putc_raw()` 和 `uart_send_str()`，用于不做 CRLF 转换的原始发送。

```

/* 发送单个字符，不进行 CRLF 转换（用于 ANSI 转义序列和帧缓冲渲染） */
static void uart_putc_raw(char c)
{
    volatile unsigned char lsr;
    do
    {
        lsr = REG8(UART_BASE + UART_LS_REG);
    } while ((lsr & UART_LS_THRE) == 0);
    REG8(UART_BASE + UART_TH_REG) = c;
}

/* 发送字符串（无 CRLF 转换） */
static void uart_send_str(const char *s)
{
    while (*s)
    {
        uart_putc_raw(*s++);
    }
}

```

在此基础上，`uart_ansi_pos(row, col)` 负责输出 `ESC[row;colH` 形式的光标定位序列。差分渲染时，只要发现某一行内容发生变化，程序就把光标定位到该行第一列，然后重新输出这一行。PuTTY 会把这些 ANSI 序列解释为终端控制命令，从而实现原地更新。

(3.1) 开发版输入模式（GPIO 模块）

此函数的核心功能是读取 GPIO 模块所有输入引脚的当前状态，也就是用户在开发板上的输入，并将这组状态作为一个 32 位的整数返回，这个 32 位值的每一位都对应一个 GPIO 引脚的当前电平状态。用户的输入其实存储在开发板中的寄存器中，我们需要从中实时读取内容即可获取输入。具体实现如下：

```

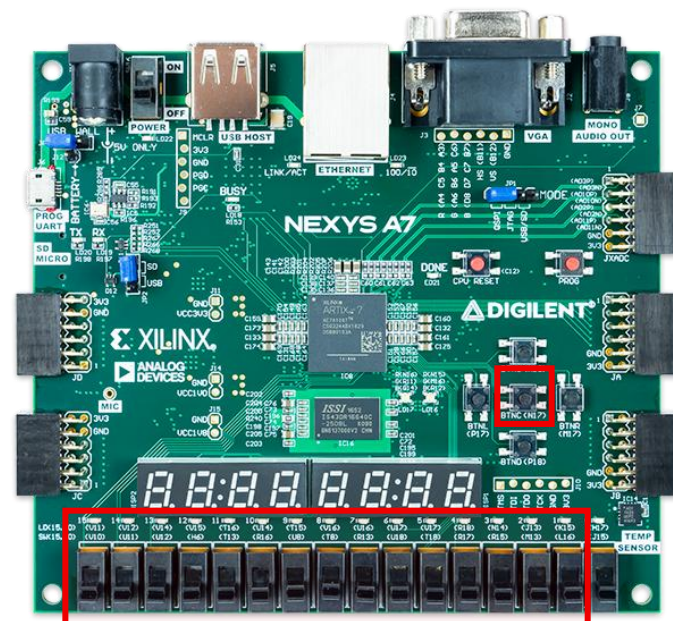
void gpio_init() /* GPIO 模块初始化函数 */
{
    REG32(GPIO_BASE + GPIO_OE_REG) = 0xffffffff; /* 所有输出端口使能*/
    REG32(GPIO_BASE + GPIO_INTE_REG) = 0x00000000; /* 禁用所有中断*/

    gpio_out(0x0f0f0f0f); /* 输出 0x0f0f0f0f*/

    /* 通过 UART 输出 GPIO 模块初始化完毕信息 */
    uart_print_str("GPIO initialize done ! \n");

    return;
}

```



0 bit

15-1bit

(3.2) 键盘输入模式

在开发版引脚输入的基础上，我新增了非阻塞 UART 输入读取。uart_getc() 先检查 UART Line Status Register 的 Data Ready 位，只有存在输入字符时才读取；没有输入时直接返回 0。这样 TaskStart 的主循环可以持续运行，不会因为等待键盘输入而卡死。

```

/* 从 UART 读取一个字符，无数据时返回 0（非阻塞） */
static char uart_getc(void)
{
    unsigned char lsr;

```

```

lsr = REG8(UART_BASE + UART_LS_REG);
if (lsr & 0x01)
{
    /* Data Ready? */
    return (char)REG8(UART_BASE + UART_TH_REG); /* 读 RBR (与 THR 同地址) */
}
return 0;
}

```

uart_get_key() 则负责把原始字符归一化为程序内部的键码。普通字母键比较简单，例如 W/A/S/D 分别映射为上下左右；数字 1/2/3/4 或 E/M/H/L 映射为四种难度；Enter 或空格映射为确认键。方向键稍复杂，因为 PuTTY 发送的是 ESC [A/B/C/D 这类多字符转义序列，所以代码用 esc_state 记录当前解析到哪一步。

```

if (esc_state == 0)
{
    if (c == 27)
    { /* ESC */
        esc_state = 1;
        return K_NONE;
    }
    /* 单字符按键 */
    if (c == 'w' || c == 'W')
        return K_UP;
    if (c == 's' || c == 'S')
        return K_DOWN;
    if (c == 'a' || c == 'A')
        return K_LEFT;
    if (c == 'd' || c == 'D')
        return K_RIGHT;
    if (c == '\r' || c == ' ')
        return K_CONFIRM; /* Enter 或 空格 */
    if (c == '1' || c == 'e' || c == 'E')
        return K_EASY;
    if (c == '2' || c == 'm' || c == 'M')
        return K_MEDIUM;
    if (c == '3' || c == 'h' || c == 'H')
        return K_HARD;
    if (c == '4' || c == 'l' || c == 'L')
        return K_HELL;
    return K_NONE;
}
else if (esc_state == 1)
{
    if (c == '[')

```

```

    {
        esc_state = 2;
        return K_NONE;
    }
    esc_state = 0; /* 非法序列，重置 */
    return K_NONE;
}
else
{ /* esc_state == 2 */
    esc_state = 0;
    if (c == 'A')
        return K_UP;
    if (c == 'B')
        return K_DOWN;
    if (c == 'C')
        return K_RIGHT;
    if (c == 'D')
        return K_LEFT;
    return K_NONE;
}
}

```

需要注意的是，键盘模式和 GPIO 模式的交互方式略有不同。GPIO 模式仍然是“拨动开关后按 N17 确认”；键盘模式在菜单阶段按 1/2/3/4 直接选择难度，在游戏阶段按 WASD 或方向键直接移动，胜利后按 Enter 或空格重开。这使 PuTTY 调试时的操作手感更加自然。

功能	GPIO 模式	键盘模式	说明
选择难度	拨动方向开关后按 N17	1/2/3/4 或 E/M/H/L	键盘选择在菜单阶段直接生效。
向上移动	UP 开关 + N17	W 或方向键 ↑	键盘方向键由 ESC [A/B/C/D 序列解析。
向下移动	DOWN 开关 + N17	S 或方向键 ↓	游戏阶段方向键直接触发移动。
向左移动	LEFT 开关 + N17	A 或方向键 ←	两种输入模式可同时保留。
向右移动	RIGHT 开关 + N17	D 或方向键 →	GPIO 仍采用确认键上升沿触发。
通关重开	N17	Enter 或 Space	胜利后进入等待重启阶段。

（4）帧缓冲与差分渲染

显示系统的核心是两个二维字符数组 `fb_new` 和 `fb_old`。`fb_new` 表示本次准备显示的新画面，`fb_old` 表示上一次已经显示到终端的旧画面。每次需要刷新界面时，程序先调用 `build_difficulty_screen()` 或 `build_game_screen()` 把目标画面写入 `fb_new`，再调用 `fb_render()` 比较新旧两帧。

```
static char fb_new[FB_ROWS][FB_COLS];
```

```
static char fb_old[FB_ROWS][FB_COLS];
static int fb_first_frame = 1;      /* 首帧需要全屏清除 + 绘制 */
static int fb_active_rows = FB_ROWS; /* 当前实际使用的行数（避免超出终端高度导致滚屏错位） */
```

首帧渲染时，程序会向 PuTTY 发送清屏和光标复位序列，然后输出所有活跃行。之后每次渲染时，fb_render() 逐行比较 fb_new 和 fb_old，如果某一行完全相同，就不输出；如果某一行发生变化，就把光标定位到该行并输出整行。虽然实现粒度是“变化行”而不是单个字符，但从终端视觉效果看，已经实现了差分更新：只有发生变化的区域被重画，窗口不会持续滚屏。

```
if (fb_first_frame)
{
    /* 首帧：清屏，然后输出活跃行。
     * 输出行数 = fb_active_rows，不会超出终端高度，避免滚屏导致后续光标定位错位 */
    uart_ansi_seq("\033[2J\033[H");
    for (r = 0; r < fb_active_rows; r++)
    {
        int c;
        for (c = 0; c < FB_COLS; c++)
        {
            uart_putc_raw(fb_new[r][c]);
        }
    }
    fb_first_frame = 0;
    any_change = 1;
}
else
{
    /* 差分更新：仅输出改变的活跃行 */
    for (r = 0; r < fb_active_rows; r++)
    {
        if (my_memcmp(fb_new[r], fb_old[r], FB_COLS) != 0)
        {
            /* 定位光标并输出整行 */
            uart_ansi_pos(r + 1, 1);
            {
                int c;
                for (c = 0; c < FB_COLS; c++)
                {
                    uart_putc_raw(fb_new[r][c]);
                }
            }
        }
        any_change = 1;
    }
}
```

```

    }
}
/* 渲染后隐藏光标 */
if (any_change)
{
    uart_ansi_seq("\033[?251");
}
}

```

这也是为什么串口程序需要从 SSCOM 切换到 PuTTY。SSCOM 更适合显示普通串口日志，不适合把 ANSI 光标定位序列解释为终端控制命令；PuTTY 则更接近真正终端，能够支持清屏、定位和隐藏光标等控制效果。迷宫这类需要原地刷新的程序，用 PuTTY 更合理。

(5) 菜单界面与游戏界面构建

build_difficulty_screen() 用于构造难度选择界面。它会设置当前帧缓冲活跃行数，然后在固定位置写入菜单标题、GPIO 选择说明和键盘选择说明。由于菜单阶段不需要显示大地图，所以活跃行数较少，终端不会产生多余滚动。

```

static void build_difficulty_screen(void)
{
    int row;

    /* 难度选择界面约 11 行 */
    fb_set_rows(13);
    fb_clear();
    row = 2;
    fb_write_str(row, 25, "=== Select Difficulty ===");
    row += 2;
    fb_write_str(row, 22, "Switch RIGHT (S1) up -> Easy (10x10)");
    row += 1;
    fb_write_str(row, 22, "Switch LEFT (S2) up -> Medium (15x15)");
    row += 1;
    fb_write_str(row, 22, "Switch DOWN (S3) up -> Hard (20x20)");
    row += 1;
    fb_write_str(row, 22, "Switch UP (S4) up -> Hell (30x30)");
    row += 2;
    fb_write_str(row, 22, "Keys: 1/E=Easy 2/M=Medium 3/H=Hard 4/L=Hell");
    row += 1;
    fb_write_str(row, 24, "Set ONE switch & press N17 to confirm...");
}

```

build_game_screen() 则根据当前 board_size 动态计算迷宫区域和提示行位置。Easy 只

需要较少行数，Hell 需要更多行数。函数会把玩家位置显示为 @，出口显示为 \$，墙壁显示为 #，通路显示为 .，并在底部显示当前状态消息和操作提示。

```
if (r == player_row && c == player_col)
{
    cell_char = '@'; /* 玩家位置 */
}
else if (r == exit_row && c == exit_col)
{
    cell_char = '$'; /* 出口位置 */
}
else if (active_map[r * board_size + c] == 1)
{
    cell_char = '#'; /* 墙壁 */
}
else
{
    cell_char = '.'; /* 通路 */
}
```

(6) 难度选择与双输入合并

TaskStart 的第一阶段是难度选择。程序先强制下一帧全量刷新，绘制菜单，然后同时监听 GPIO 和键盘。GPIO 分支沿用旧版本逻辑：读取 gpio_in()，用最低位作为 N17 确认键，用 choice 的低四位判断方向开关。键盘分支则检查 uart_get_key() 返回的 K_EASY/K_MEDIUM/K_HARD/K_HELL。

```
if (ready && !last_ready) {
    if (choice & 0x00000001) { active_map = maze_easy; ... }
    else if (choice & 0x00000002) { active_map = maze_medium; ... }
    else if (choice & 0x00000004) { active_map = maze_hard; ... }
    else if (choice & 0x00000008) { active_map = maze_hell; ... }
}

if (!selected && key != K_NONE && key != last_key) {
    if (key == K_EASY) { active_map = maze_easy; ... }
    else if (key == K_MEDIUM) { active_map = maze_medium; ... }
}
```

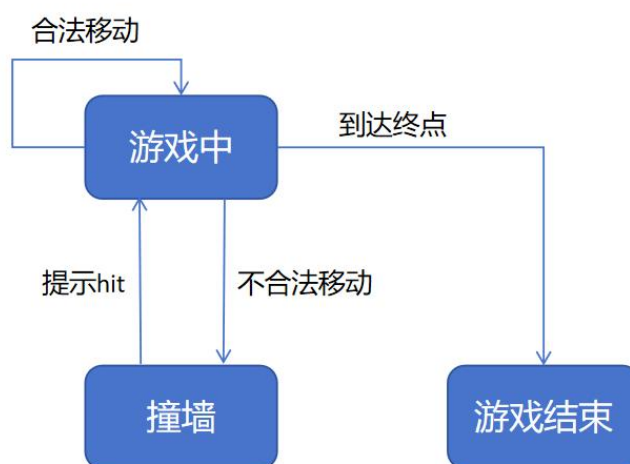
这里的 last_ready 和 last_key 分别用于避免 GPIO 与键盘重复触发。last_ready 检测 N17 上升沿，last_key 则避免同一个键码在多次轮询中被连续处理。这样一来，菜单选择阶段既能兼容实验板开关，又能兼容 PuTTY 键盘。

难度	地图规模	GPIO 选择	键盘选择	默认出口坐标
----	------	---------	------	--------

难度	地图规模	GPIO 选择	键盘选择	默认出口坐标
Easy	10 x 10	S1 + N17	1 / E	(8, 8)
Medium	15 x 15	S2 + N17	2 / M	(13, 13)
Hard	20 x 20	S3 + N17	3 / H	(18, 18)
Hell	30 x 30	S4 + N17	4 / L	(28, 28)

(7) 状态转换图与代码解释

按照游戏实际运行逻辑，游戏状态包括：游戏中、撞墙提示和成功。玩家处于游戏中时，如果移动到通路，就仍然保持游戏中；如果目标位置是墙壁，则进入“撞墙提示”状态，显示 Oops! 后立即重新渲染并返回游戏中；如果玩家到达出口，则进入成功状态，显示 You Win 并等待重开。



代码中通过 `game_won` 与 `status_msg` 两个变量体现状态。`game_won` 控制是否退出游戏循环；`status_msg` 控制当前帧底部显示什么提示。正常移动时 `status_msg` 为空，撞墙时为 `Oops! Hit a wall.`，到达终点后为 `You Win!`

```

if (active_map[next_r * board_size + next_c] == 0)
{
    player_row = next_r;
    player_col = next_c;
    if (player_row == exit_row && player_col == exit_col)
    {
        game_won = 1;
        status_msg = "You Win!";
    }
    else
    {
        status_msg = (char *)0;
    }
}

```

```

    }
    build_game_screen(diff_name, status_msg);
    fb_render();
}
else
{
    status_msg = "Oops! Hit a wall.";
    build_game_screen(diff_name, status_msg);
    fb_render();
}

```

撞墙状态之所以说“提示后直接返回游戏中”，是因为撞墙并不会改变玩家坐标，也不会跳出 `while (!game_won)` 主循环。程序只是把 `status_msg` 改成 `Oops`，重建一帧游戏界面并差分渲染，下一轮循环仍继续等待移动输入。成功状态则不同：`game_won` 被置为 1 后会结束游戏循环，随后进入等待重启阶段。

（8）移动判定与键盘/GPIO 共存

游戏运行阶段同样同时监听 GPIO 和键盘。GPIO 模式下，只有检测到 N17 上升沿时才根据开关组合产生一次移动请求；键盘模式下，只要读取到新的方向键码，就直接产生移动请求。两种输入来源最终都会归一化为 `move_up`、`move_down`、`move_left`、`move_right` 四个方向标志，再统一计算 `next_r` 和 `next_c`。

```

if (ready && !last_ready)
{
    do_move = 1;
    move_up = ((choice & 0x0000000F) == 0x00000008);
    move_down = ((choice & 0x0000000F) == 0x00000004);
    move_left = ((choice & 0x0000000F) == 0x00000002);
    move_right = ((choice & 0x0000000F) == 0x00000001);
}

if (!do_move && key != K_NONE && key != last_key)
{
    if (key == K_UP)
    {
        do_move = 1;
        move_up = 1;
    }
    else if (key == K_DOWN)
    {
        do_move = 1;
        move_down = 1;
    }
}

```

```

    }
    else if (key == K_LEFT)
    {
        do_move = 1;
        move_left = 1;
    }
    else if (key == K_RIGHT)
    {
        do_move = 1;
        move_right = 1;
    }
}

```

这种“先归一化、再处理”的方式让后续移动判定不需要关心输入来源。无论玩家使用实验板开关还是 PuTTY 键盘，真正改变坐标的代码都是同一套逻辑，减少了重复实现，也降低了两个输入模式行为不一致的风险。

（9）通关与重开

当玩家坐标与出口坐标重合时，`game_won` 置为 1，程序跳出游戏循环，底部状态消息改为 `You Win! Press N17 to restart...`。重启阶段也支持双输入：GPIO 模式下按 N17，键盘模式下按 Enter 或空格，都可以让 `restart` 置为 1，从而返回最外层 `for(;;)` 循环并重新进入难度选择界面。

```

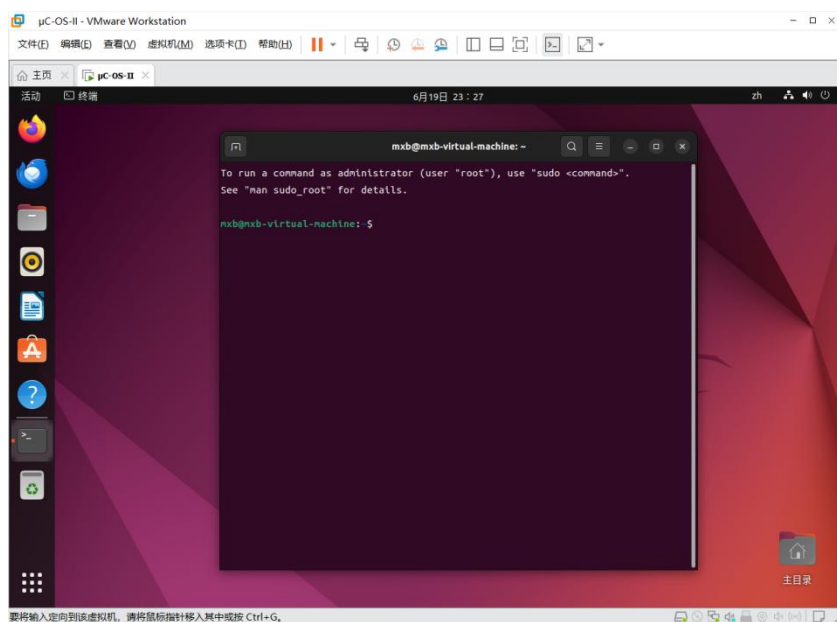
if ((ready && !last_ready) ||
    (key != K_NONE && key != last_key && key == K_CONFIRM))
{
    restart = 1;
}

```

四、交叉编译阶段

4.1 Linux-Ubuntu 环境配置

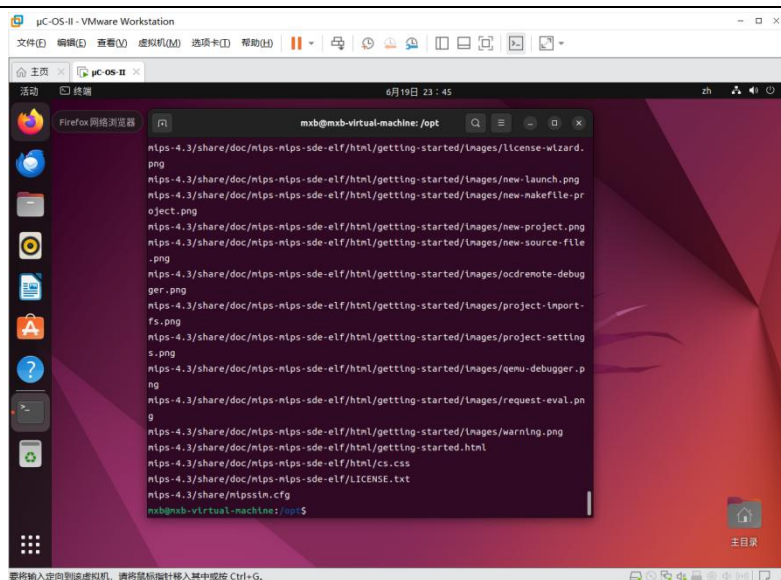
这里我是用 VMware 虚拟机，在其中安装 Linux 系统，使用 Ubuntu 的光盘映像文件进行安装；参考的是 CSDN 教程：[VMware 虚拟机安装 Ubuntu 教程\(超详细\)](#)



4.2 MIPS 编译环境的建立

首先需要安装 GNU 工具链。《自己动手写 CPU》一书的附件中提供了安装文件，mips-sde-elf-i686-pc-linux-gnu.tar.tar，将安装文件复制到 Ubuntu 的 /opt 目录下，打开 Ubuntu 的终端，使用如下命令解压缩：

```
cd /opt
tar vfxj mips-sde-elf-i686-pc-linux-gnu.tar.tar
```



然后打开用户主目录 Home 文件夹，在窗口菜单栏中选择 View->Show Hidden Files，以显示所有文件，这样可以找到一个隐藏文件.bashrc，在此文件的最后加入 PATH 的设置，如下。

```
export PATH="$PATH:/opt/mips-4.3/bin"
```

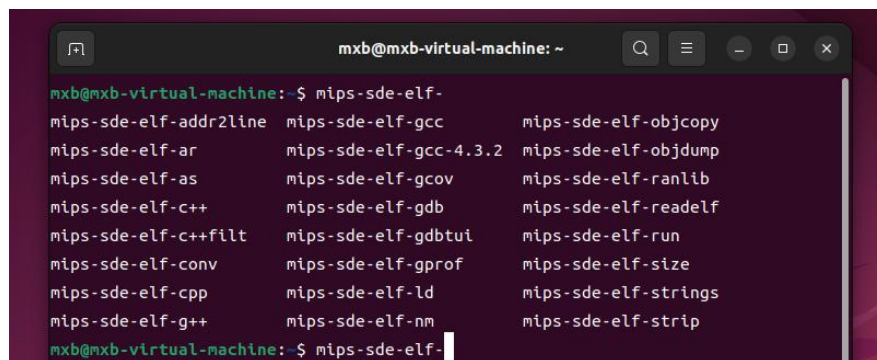
还需要安装 lib32z1、libc6-dev-i386 库

```
sudo dpkg --add-architecture i386
```

```
sudo apt update
```

```
sudo apt install libc6:i386 libstdc++6:i386 zlib1g:i386 libncurses5:i386 -y
```

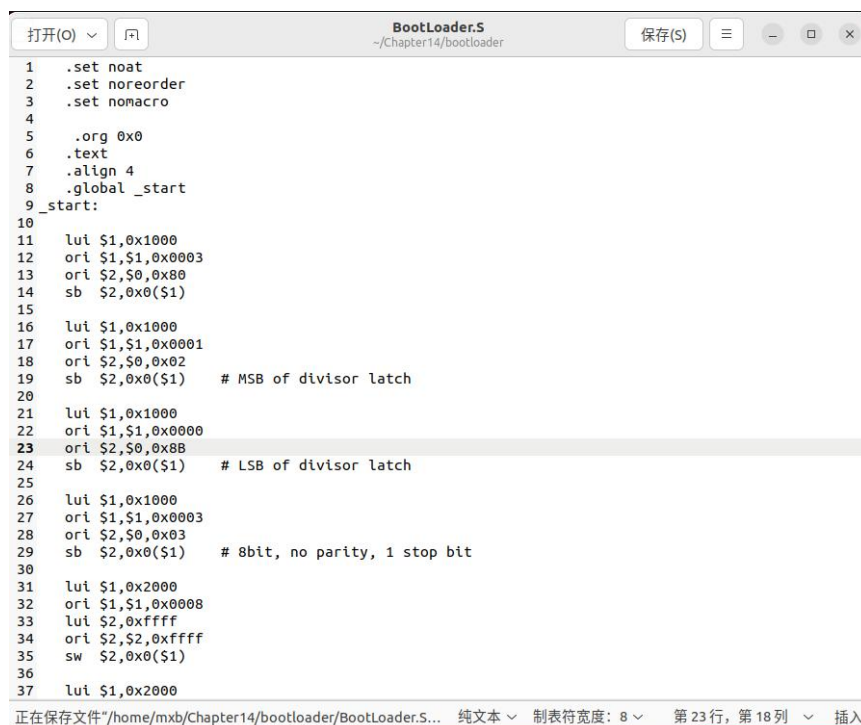
重新启动 Ubuntu 系统。重启后,打开终端,在其中输入 mips-sde-elf-, 然后按两次 Tab 键, 会列出刚刚安装的, 针对 MIPS 平台的所有编译工具, 如下图所示, 表示 GNU 工具链安装成功。



```
mxb@mxv-virtual-machine: ~  
mxb@mxv-virtual-machine:~$ mips-sde-elf-  
mips-sde-elf-addr2line  mips-sde-elf-gcc          mips-sde-elf-objcopy  
mips-sde-elf-ar         mips-sde-elf-gcc-4.3.2    mips-sde-elf-objdump  
mips-sde-elf-as         mips-sde-elf-gcov          mips-sde-elf-ranlib  
mips-sde-elf-c++        mips-sde-elf-gdb          mips-sde-elf-readelf  
mips-sde-elf-c++filt    mips-sde-elf-gdbtui       mips-sde-elf-run  
mips-sde-elf-conv       mips-sde-elf-gprof        mips-sde-elf-size  
mips-sde-elf-cpp        mips-sde-elf-ld           mips-sde-elf-strings  
mips-sde-elf-g++        mips-sde-elf-nm           mips-sde-elf-strip  
mxb@mxv-virtual-machine:~$ mips-sde-elf-
```

4.3 对 μ C/OS-II 系统进行改写、编译

首先需要修改 BootLoader 程序并重新编译, 主要是修改 UART 的频率。修改如下:



```
BootLoader.S  
~/Chapter14/bootloader  
保存(S)  
1 .set noat  
2 .set noreorder  
3 .set nomacro  
4  
5 .org 0x0  
6 .text  
7 .align 4  
8 .global _start  
9 _start:  
10  
11 lui $1,0x1000  
12 ori $1,$1,0x0003  
13 ori $2,$0,0x80  
14 sb $2,0x0($1)  
15  
16 lui $1,0x1000  
17 ori $1,$1,0x0001  
18 ori $2,$0,0x02  
19 sb $2,0x0($1) # MSB of divisor latch  
20  
21 lui $1,0x1000  
22 ori $1,$1,0x0000  
23 ori $2,$0,0x8B  
24 sb $2,0x0($1) # LSB of divisor latch  
25  
26 lui $1,0x1000  
27 ori $1,$1,0x0003  
28 ori $2,$0,0x03  
29 sb $2,0x0($1) # 8bit, no parity, 1 stop bit  
30  
31 lui $1,0x2000  
32 ori $1,$1,0x0008  
33 lui $2,0xffff  
34 ori $2,$2,0xffff  
35 sw $2,0x0($1)  
36  
37 lui $1,0x2000  
正在保存文件"/home/mxb/Chapter14/bootloader/BootLoader.S... 纯文本 制表符宽度: 8 第 23 行, 第 18 列 插入
```

执行 make all 对 BootLoader 进行编译，生成可执行对象文件 BootLoader.o，将其转为可执行的汇编代码 BootLoader.asm（在 makeall 中已实现）

```
mxh@mxh-virtual-machine:~/Chapter14/bootloader$ make all
mips-sde-elf-as -mips32 BootLoader.S -o BootLoader.o
BootLoader.S: Assembler messages:
BootLoader.S:57: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:58: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:88: Warning: Macro instruction expanded into multiple instructions
BootLoader.S:89: Warning: Macro instruction expanded into multiple instructions
mips-sde-elf-ld -T ram.ld BootLoader.o -o BootLoader.om
mips-sde-elf-objcopy -O binary BootLoader.om BootLoader.bin
mips-sde-elf-objdump -D BootLoader.om > BootLoader.asm
```

然后进行 μ C/OS-II 系统的编译，主要用的是《自己动手写 CPU》中提供的系统，在此基础上修改 UART 频率，修改如下：

```
打开(O) 保存(S)
*openmips.h
~/Chapter15/ucosII_OpenMIPS/include

1 #define REG8(add) *((volatile INT8U *) (add))
2 #define REG16(add) *((volatile INT16U *) (add))
3 #define REG32(add) *((volatile INT32U *) (add))
4
5 #define IN_CLK 100000000 /* 输入时钟是27MHz */
6
7 /*****
8
9 串口相关参数、函数
10
11 *****/
12
13 #define UART_BAUD_RATE 9600 /* 串口速率是9600bps */
14 #define UART_BASE 0x10000000
15 #define UART_LC_REG 0x00000003 /* Line Control Register */
16 #define UART_IE_REG 0x00000001 /* Interrupt Enable Register */
17 #define UART_TH_REG 0x00000000 /* Transmitter Holding Register */
18 #define UART_LS_REG 0x00000005 /* Line Status Register */
19 #define UART_DL1_REG 0x00000000 /* Divisor Latch Byte 1(LSB) */
20 #define UART_DL2_REG 0x00000001 /* Divisor Latch Byte 2(MSB) */
```

编译如下：

```
活动 终端
6月20日 00:58
mxh@mxh-virtual-machine: ~/Chapter15/ucosII_OpenMIPS

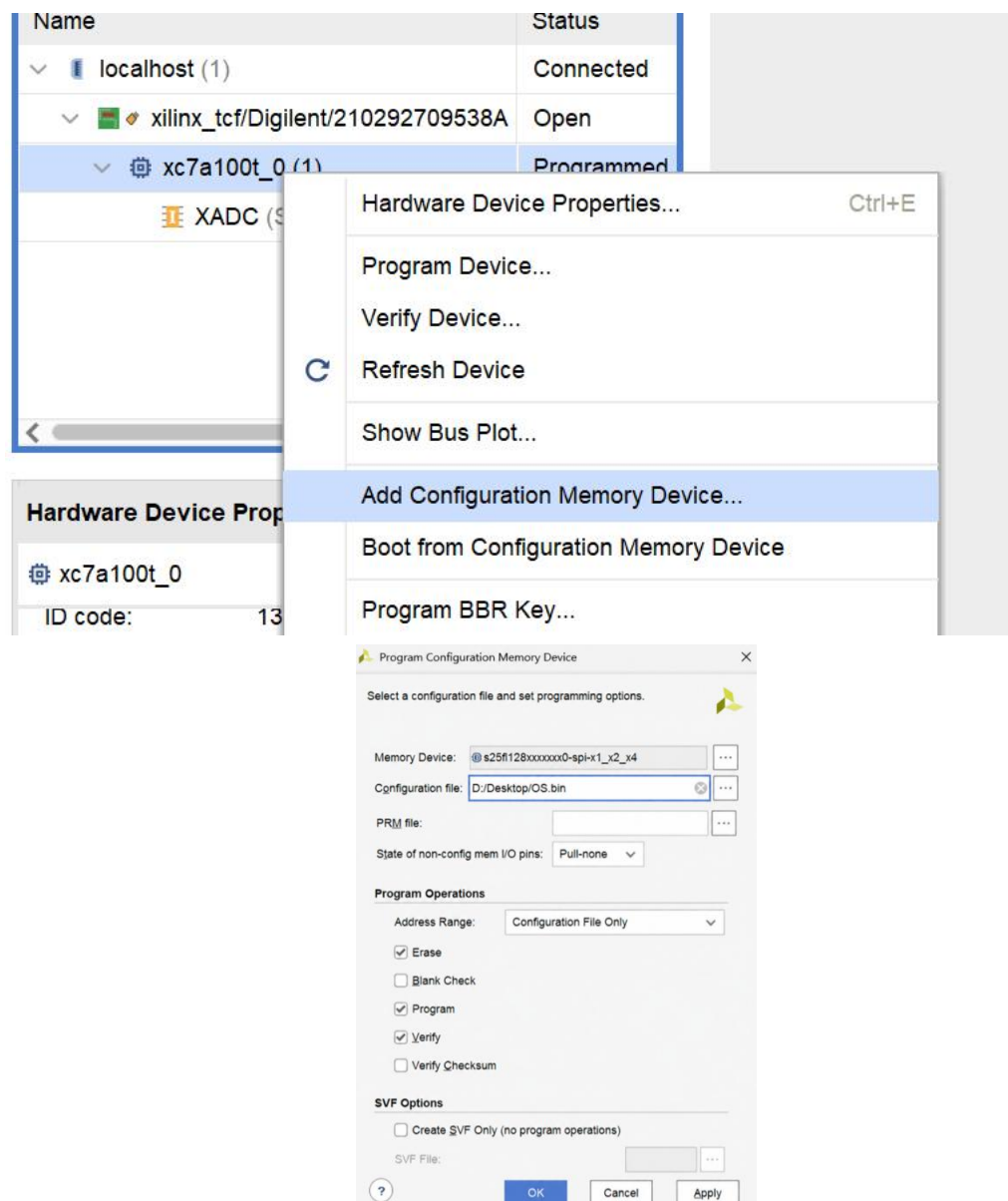
make[1]: 离开目录"/home/mxh/Chapter15/ucosII_OpenMIPS/port"
make[1]: 进入目录"/home/mxh/Chapter15/ucosII_OpenMIPS/common"
mips-sde-elf-gcc -I/home/mxh/Chapter15/ucosII_OpenMIPS/include -I/home/mxh/Chapter15/ucosII_OpenMIPS/ucos -I/home/mxh/Chapter15/ucosII_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o openmips.o openmips.c
openmips.c:144: warning: return type of 'main' is not 'int'
mips-sde-elf-ld -r -o common.o openmips.o
make[1]: 离开目录"/home/mxh/Chapter15/ucosII_OpenMIPS/common"
make[1]: 进入目录"/home/mxh/Chapter15/ucosII_OpenMIPS/ucos"
make[1]: 对"all"无需做任何事。
make[1]: 离开目录"/home/mxh/Chapter15/ucosII_OpenMIPS/ucos"
make[1]: 进入目录"/home/mxh/Chapter15/ucosII_OpenMIPS/port"
UbuntuSoftware: -I/home/mxh/Chapter15/ucosII_OpenMIPS/include -I/home/mxh/Chapter15/ucosII_OpenMIPS/ucos -I/home/mxh/Chapter15/ucosII_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o os_cpu_c.o os_cpu_c.c
os_cpu_c.c: In function 'BSP_Exception_Handler':
os_cpu_c.c:372: warning: unused variable 'EPC'
mips-sde-elf-gcc -I/home/mxh/Chapter15/ucosII_OpenMIPS/include -I/home/mxh/Chapter15/ucosII_OpenMIPS/ucos -I/home/mxh/Chapter15/ucosII_OpenMIPS/common -Wall -Wstrict-prototypes -Werror-implicit-function-declaration -fomit-frame-pointer -fno-strength-reduce -O2 -g -pipe -fno-builtin -nostdlib -mips32 -c -o os_cpu_a.o os_cpu_a.c
mips-sde-elf-ld -r -o port.o os_cpu_c.o os_cpu_a.o
make[1]: 离开目录"/home/mxh/Chapter15/ucosII_OpenMIPS/port"
mips-sde-elf-gcc -Tram.ld -o ucosiI.om common.o ucoc/ucos.o port/port.o -nostdlib -lgcc -e 256
mips-sde-elf-objcopy -O binary ucosiI.om ucosiI.bin
mips-sde-elf-objdump -D ucosiI.om > ucosiI.asm
./BinMerge.exe -f ucosiI.bin -o OS.bin
mxh@mxh-virtual-machine: ~/Chapter15/ucosII_OpenMIPS$
```

得到二进制文件，将 OS.bin 写入板子上的 Flash，当 OpenMIPS 运行时，会首先运行 BootLoader，后者将 μ C/OC-的代码复制到 SDRAM 中，然后跳转到 SDRAM，把控制权交给 μ C/OS-II，于是 μ C/OS-II 就运行起来了。

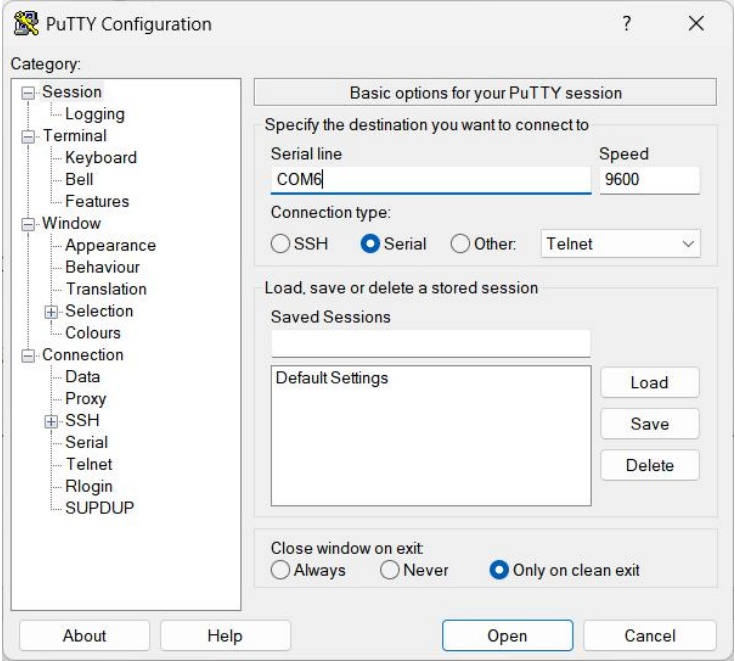
五、实验结果

5.1 串口通讯与可视化界面

和实验二一样，综合、实现、生成比特流完成之后，在下板之前还需要把之前在 Ubuntu 中编译好的 OS.bin 文件载入。在 hardware 中开发版右键选择 Add Configuration Memory Device。



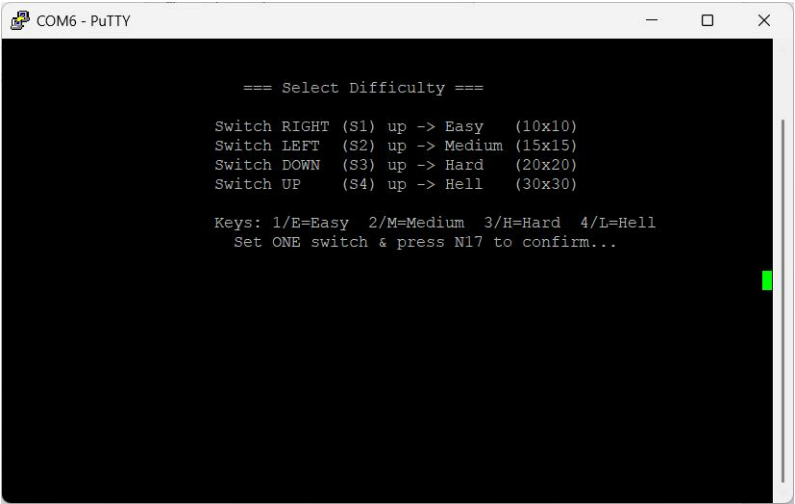
打开 PuTTY 串口调试程序，输入对应的端口号（COM6）和波特率（9600），然后打开，将开发版右侧第一个开关上拨，即可看到游戏成功启动。

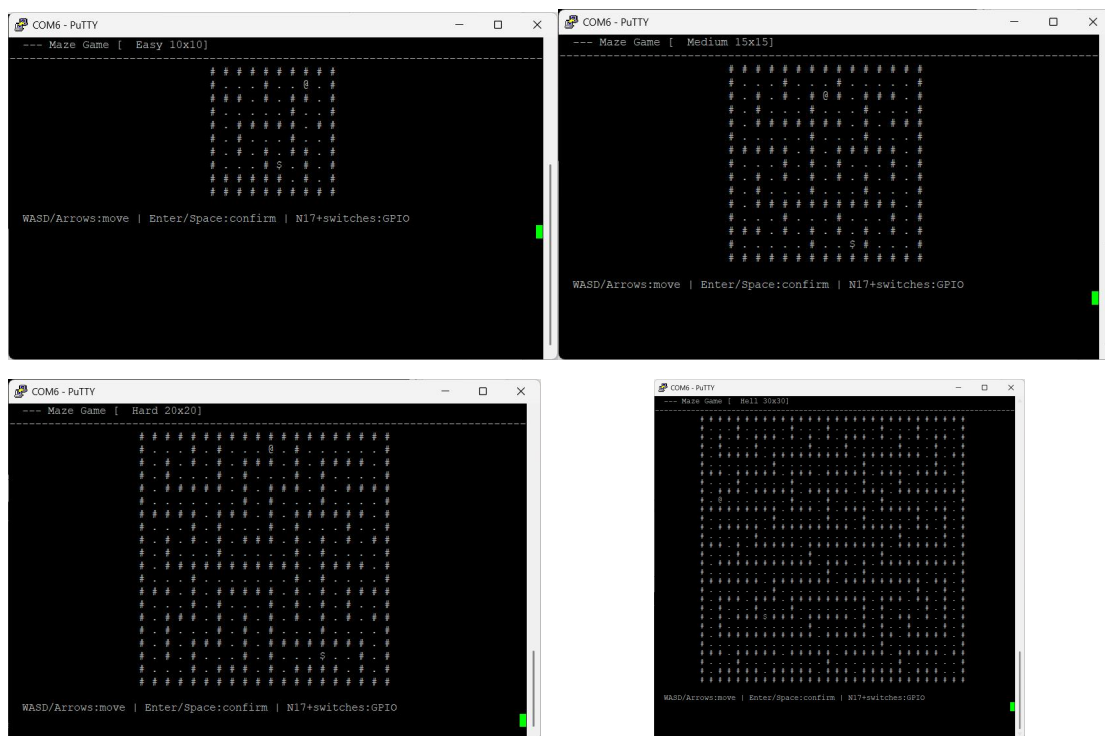


5.2 游戏流程

（1）难度可选择

游戏共有四个难度选择，可以用键盘 1234 或者开发版引脚选择：



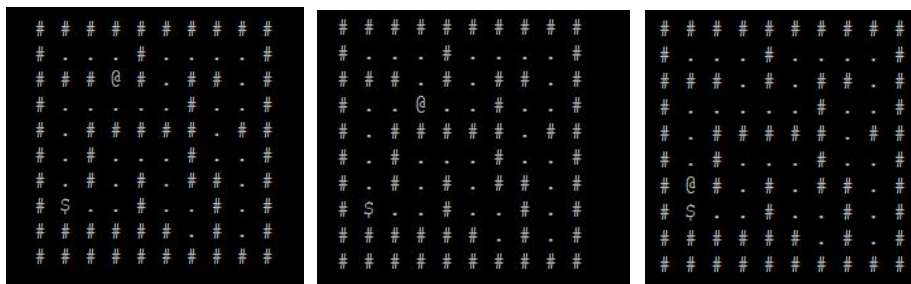


PuTTY 显示效果：在 PuTTY 中运行时，迷宫界面会保持在固定位置显示。初次进入菜单或游戏时终端清屏，之后每次移动只会更新发生变化的行。相比 SSCOM 中不断追加输出的显示效果，PuTTY 下的界面更接近真实终端游戏，玩家不会因为每一步重画整张地图而失去当前位置。

（2）键盘、开发版移动模式

在 PuTTY 中可以直接使用 1/2/3/4 选择难度，进入迷宫后用 WASD 或方向键移动。测试时可分别验证普通字母键和方向键：W 与 ↑ 都应表现为向上移动，S 与 ↓ 表现为向下移动，A 与 ← 表现为向左移动，D 与 → 表现为向右移动。通关后按 Enter 或空格应返回难度选择。

键盘模式使调试效率明显提高。过去使用 GPIO 开关时，每一步都需要拨动开关并按 N17；现在使用键盘可以直接连续移动，更适合验证差分渲染、撞墙提示和通关路径。



(3) 撞墙与成功状态测试

当玩家向墙壁方向移动时，地图中 @ 的位置保持不变，底部状态行显示 `Oops! Hit a wall.`。下一次合法移动后，`status_msg` 被清空，界面恢复普通游戏中状态。该现象说明撞墙状态只是一个短暂提示状态，并不会中断游戏循环。

```
      # # # # # # # # #
      # . . . # . . . #
      # # # . # . # . #
      # . . . . # . . #
      # . # # # # . # #
      # . # . . . # . #
      # @ # . # . # . #
      # $ . . # . . #
      # # # # # . # . #
      # # # # # # # # #

Oops! Hit a wall.
WASD/Arrows:move | Enter/Space:confirm | N17+switches:GPIO
```

当玩家到达 \$ 终点时，底部显示 `You Win! Press N17 to restart...`，程序进入成功后的等待重开阶段。此时继续按方向键不会再移动迷宫，只有 `N17`、`Enter` 或 `Space` 会重新开始。

```
      # # # # # # # # #
      # . . . # . . . #
      # # # . # . # . #
      # . . . . # . . #
      # . # # # # . # #
      # . # . . . # . . #
      # . # . # . # . #
      # @ . . # . . #
      # # # # # . # . #
      # # # # # # # # #

You Win! Press N17 to restart...
```

五、实验总结

5.1 遇到的问题

第一，SSCOM 不适合作为本版本的最终显示终端。迷宫程序每一步都要刷新界面，如果串口工具只把输出当作普通日志追加显示，那么玩家会看到大量重复地图。切换到 PuTTY 后，ANSI 光标定位和清屏序列能够正常工作，差分渲染才有实际意义。

第二，键盘方向键不是单字符输入，而是 ESC 开头的多字符序列。若只按普通字符处理，就无法识别方向键。因此代码使用 `esc_state` 保存解析状态，分三步处理 ESC、[和最终的 A/B/C/D。这个过程让我更清楚地理解了终端键盘输入并不等同于简单字符读取。

第三，差分渲染需要维护新旧两份画面。旧版本每次调用 `print_board()` 直接输出即可；

新版本必须先在 `fb_new` 中构造完整画面，再与 `fb_old` 比较，最后复制新帧到旧帧。虽然代码结构更复杂，但换来的显示效果更接近真正的交互程序。

第四，双输入模式需要避免重复触发。GPIO 使用 `last_ready` 判断 N17 上升沿，键盘使用 `last_key` 过滤连续读取到的同一键码。否则一次按键可能在高速轮询中被处理多次，导致角色移动过头或菜单选择重复。

5.2 心得体会

这次实验让我对“串口程序”有了更具体的理解。串口并不只能打印日志，只要终端支持 ANSI 控制序列，也可以构造出较完整的字符界面。PuTTY、原始 UART 输出、光标定位和帧缓冲配合起来，就能把一个原本不断滚屏的程序改造成原地更新的小游戏界面。

键盘模式的加入也让我意识到，嵌入式实验并不一定只能通过开发板物理输入操作。只要 UART 同时具备收发能力，就可以把 PC 端终端键盘作为另一种输入设备。这样既保留了硬件实验特色，又提升了调试效率。

从整体上看，迷宫小游戏已经包含输入解析、状态转换、差分渲染、地图管理和任务循环等多个模块。这个过程很好地体现了系统实验的价值：一个小功能背后，往往连接着硬件寄存器、操作系统任务、终端协议和程序结构设计。

参考资料

- [1] 秦国锋, 王力生, 陆有军, 郭玉臣. 计算机系统结构实验指导. 清华大学出版社, 2019.
- [2] 雷思磊. 自己动手写 CPU. 电子工业出版社, 2014.
- [3] 张晨曦, 王志英等. 计算机体系结构. 高等教育出版社, 2014.